```
}
```

τότε μόνο οι κλάσεις που ανήκουν στο ίδιο πακέτο με την Rectangle θα μπορούν να τον καλέσουν και άρα μόνο αυτές θα μπορούν να μεταβάλουν το πλάτος ενός ορθογώνιου μετά τη δημιουργία του.
Τα μέλη-δεδομένα έχουν χαρακτηριστεί ως private. Αυτό σημαίνει πως επιτρέπεται η προσπέλασή τους μόνο μέσα στην κλάση Rectangle. Αν ωστόσο κάποιος χρήστης της κλάσης επιθυμεί να διαβάσει ή να γράψει τα μέλη αυτά μπορεί να χρησιμοποιήσει τους δημόσιους ρυθμιστές και αναγνώστες.
Τέλος, υπάρχει και ο προσδιοριστής protected που σχετίζεται με την κληρονομικότητα και παρουσιάζεται στην ενότητα 13.1.2.

11.4 Η λέξη κλειδί this

Προσέξτε πως οι μη στατικές συναρτήσεις καλούνται αναγκαστικά από ένα στιγμιότυπο της κλάσης. Μερικές φορές μέσα στον ορισμό μιας μη στατικής συνάρτησης είναι δυνατόν να χρειάζεται να αναφερθούμε στο στιγμιότυπο που την καλεί. Βεβαίως η συνάρτηση μπορεί να κληθεί από πολλά διαφορετικά στιγμιότυπα, κανένα εκ των οποίων δεν είναι γνωστό κατά τον χρόνο που αναπτύσσουμε τη συνάρτηση. Για τον λόγο αυτόν, η Java περνάει μια αναφορά στο στιγμιότυπο που κάλεσε τη συνάρτηση μέσω της λέξης-κλειδί this.

Σημειώστε πως επειδή οι στατικές συναρτήσεις δεν καλούνται υποχρεωτικά από αντικείμενο της κλάσης, δεν επιτρέπεται η χρήση του this μέσα σε στατική συνάρτηση.

Ακολουθεί παράδειγμα χρήσης της λέξης-κλειδί this.

```
public void setLength(double length) {
     if (length<=0)
          throw new IllegalArgumentException();
     this.length = length;
}
```

Η παράμετρος στη συνάρτηση setLength έχει ίδιο όνομα με τη μεταβλητή στιγμιότυπου length. Η παράμετρος λειτουργεί σαν τοπική μεταβλητή και αποκρύπτει τη μεταβλητή length του στιγμιότυπου. Το αναγνωριστικό length μέσα στην setLength αναφέρεται στην παράμετρο. Αν θέλουμε μέσα στην setLength να αναφερθούμε στη μεταβλητή του στιγμιότυπου μπορούμε να χρησιμοποιήσουμε το this όπως φαίνεται στον παραπάνω κώδικα.

Ανάλογα με τα συμφραζόμενα (context), η λέξη-κλειδί this, μπορεί επιπλέον να χρησιμοποιηθεί για την κλήση ενός δημιουργού της κλάσης μέσα σε έναν άλλο δημιουργό.

```
public Rectangle(double len) {
     this(length,length);
}
```

Εδώ ξαναγράφουμε τον δημιουργό κατασκευής τετραγώνων. Το τετράγωνο δημιουργείται με κλήση στον δημιουργό που έχει δύο παραμέτρους και επιτυγχάνεται με χρήση του this. Ας σημειωθεί πως τέτοια κλήση μπορεί να γίνει μόνο στην πρώτη γραμμή ενός δημιουργού.

11.5 Εμβέλεια και χρόνος ζωής

Τα στατικά μέλη της κλάσης φορτώνονται στη μνήμη συνήθως όταν η εφαρμογή χρειασθεί την κλάση στην οποία ανήκουν. Στη συνέχεια διατηρούνται μέχρι το τέλος της εφαρμογής. Επομένως, αυτά τα όρια διαμορφώνουν τον χρόνο ζωής τους. Αντίθετα, τα μέλη στιγμιότυπου δημιουργούνται ταυτόχρονα με το στιγμιότυπο στο οποίο ανήκουν και καταστρέφονται μαζί με αυτό.

Η εμβέλεια των μελών της κλάσης καθορίζεται από τον προσδιοριστή προσπέλασης του μέλους σε συνδυασμό με τον προσδιοριστή προσπέλασης της κλάσης. Για παράδειγμα, ένα μέλος που ανήκει σε δημόσια κλάση και είναι δημόσιο και το ίδιο μπορεί να προσπελαστεί από οποιονδήποτε χρήστη της κλάσης. Ένα μέλος όμως που ανήκει σε δημόσια κλάση αλλά είναι το ίδιο ιδιωτικό μπορεί να προσπελαστεί μόνο από την κλάση

στην οποία ανήκει. Παρόμοια, ένα μέλος που είναι δημόσιο αλλά ανήκει σε κλάση με πρόσβαση πακέτου μπορεί να προσπελαστεί μόνο από κλάσεις του ίδιου πακέτου.

11.6 Άλλες αναγκαίες συναρτήσεις της κλάσης

Προκειμένου η κλάση μας να είναι ολοκληρωμένη θα πρέπει να διαθέτει ως μέλη τρεις ακόμη συναρτήσεις, την toString, την equals και την hashCode.

11.6.1 Η συνάρτηση toString

Αν προσπαθήσουμε να εκτυπώσουμε ένα αντικείμενο της κλάσης Rectangle, θα λάβουμε ως έξοδο μια περίεργη εκ πρώτης όψεως αλφαριθμητική σειρά. Για παράδειγμα, ο ακόλουθος κώδικας

```
Rectangle k=new Rectangle(3, 6);
System.out.println(k);
```

θα τυπώσει κάτι σαν PackageName.Rectangle@4769b07b.
Πρόκειται για το όνομα της κλάσης, ακολουθούμενο από τον κωδικό κατακερματισμού του στιγμιότυπου k εκφρασμένο σε δεκαεξαδική μορφή. Στις περισσότερες περιπτώσεις όμως που επιχειρούμε να τυπώσουμε ένα στιγμιότυπο, εκείνο που θέλουμε είναι η τιμή του στιγμιότυπου και όχι το όνομα της κλάσης συνοδευόμενο από τον κωδικό κατακερματισμού. Η Java μας δίνει τη δυνατότητα να το πετύχουμε ορίζοντας κατάλληλα τη συνάρτηση toString().

```
@Override
public String toString() {
     return "Rectangle{" + "length=" + length + ", width=" +
     width + '}';
}
```

Η συνάρτηση toString είναι υποχρεωτικά public και επιστρέφει μια αναπαράσταση του στιγμιότυπου που την καλεί με τη μορφή αλφαριθμητικής σειράς. Επιπλέον, από τον ορισμό της συνάρτησης προηγείται η ειδική σημείωση (annotation) @Override. Η σημείωση αυτή δεν είναι υποχρεωτική από τον μεταγλωττιστή, ωστόσο είναι χρήσιμη και δεν θα πρέπει να ξεχνάμε να την προσθέτουμε. Το νόημά της σχετίζεται με την κληρονομικότητα και το παρουσιάζουμε στην ενότητα 13.1.1. Στο σώμα της συνάρτησης toString βάζουμε τον κώδικα που καθορίζει τι θα παίρνουμε σε όλες τις περιπτώσεις που χρησιμοποιούμε ένα στιγμιότυπο της κλάσης αλλά χρειάζεται μια αλφαριθμητική αναπαράστασή του. Έτσι τώρα, η εκτύπωση του αντικειμένου k θα τυπώσει Rectangle{length=3.0, width=6.0}. Η συνάρτηση toString() καλείται αυτόματα (implicitly) όπου είναι απαραίτητο, αλλά μπορεί να κληθεί και σαφώς (explicitly) από την εφαρμογή.

11.6.2 Η συνάρτηση equals

Μια άλλη αναγκαία συνάρτηση είναι η equals. Όπως έχουμε αναφέρει, τα στιγμιότυπα συνιστούν μεταβλητές αναφοράς. Επομένως, η σύγκριση μεταξύ δύο στιγμιότυπων με τον τελεστή == (equality operator) συγκρίνει τις διευθύνσεις τους και όχι τις τιμές τους. Έστω ο κώδικας

```
Rectangle r1=new Rectangle(2,6), r2=new Rectangle(2,6);
System.out.println(r1==r2);
```

Παρόλο που r1 και r2 είναι 2 ορθογώνια με ίδιες διαστάσεις, ο έλεγχος ισότητας εδώ μας επιστρέφει false. Πράγματι, έχουμε 2 διαφορετικά στιγμιότυπα. Η μεταβλητή r1 αναφέρεται στη μνήμη που έχει κατανεμηθεί για το ένα ορθογώνιο και η r2 στη μνήμη του άλλου. Επομένως, το αποτέλεσμα false είναι αναμενόμενο. Πώς όμως μπορούμε να συγκρίνουμε τα ίδια τα στιγμιότυπα και όχι τις διευθύνσεις τους; Εδώ έρχεται η συνάρτηση equals να δώσει τη λύση.

```
@Override
```

```
public boolean equals(Object obj) {
     if (this == obj) {
          return true;
     }
     if (obj == null) {
          return false;
     }
     if (getClass() != obj.getClass()) {
          return false;
     }
     final Rectangle other = (Rectangle) obj;
     if (Double.compare(width, other.width) != 0) {
          return false;
     }
     return Double.compare(length, other.length) == 0;
}
```

Η equals είναι αναγκαστικά public, επιστρέφει boolean και διαθέτει μια τυπική παράμετρο τύπου Object. Επιπλέον, η equals πρέπει να πληροί τις ακόλουθες ιδιότητες [1]:

1. Ανακλαστική: για οποιαδήποτε μη-μηδενική τιμή αναφοράς x, η έκφραση x.equals (x) πρέπει να επιστρέφει true.
2. Συμμετρική: για οποιεσδήποτε μη μηδενικές τιμές αναφοράς x και y, x.equals (y) πρέπει να επιστρέφει true εάν και μόνο εάν το y.equals (x) επιστρέφει true.
3. Μεταβατική: για οποιεσδήποτε μη μηδενικές τιμές αναφοράς x, y και z, εάν x.equals (y) επιστρέφει true και y.equals (z) επιστρέφει true, τότε x.equals (z) θα πρέπει να επιστρέψει true.
4. Για οποιεσδήποτε μη-μηδενικές τιμές αναφοράς x και y, όλες οι κλήσεις του x.equals (y) επιστρέφουν σταθερά true ή σταθερά false, υπό την προϋπόθεση ότι x και y παραμένουν σταθερά.
5. Για οποιαδήποτε μη-μηδενική τιμή αναφοράς x, x.equals (null) θα πρέπει να επιστρέφει false.

Μέσα στην equals αρχικά γίνεται έλεγχος αν το στιγμιότυπο που κάλεσε την equals είναι το ίδιο με την παράμετρό της. Σε αυτήν την περίπτωση επιστρέφεται true. Στη συνέχεια ελέγχεται αν η παράμετρος είναι null. Αν είναι, με δεδομένο ότι το στιγμιότυπο που κάλεσε την equals δεν είναι null, επιστρέφεται false. Μετά ελέγχεται αν η παράμετρος ανήκει στην ίδια κλάση με το στιγμιότυπο που κάλεσε την equals. Αν τα δύο αντικείμενα ανήκουν σε διαφορετικές κλάσεις, επιστρέφεται false. Στο επόμενο βήμα ελέγχονται οι μεταβλητές στιγμιότυπου των δύο αντικειμένων μια προς μια για ισότητα. Αν κάποιος από τους ελέγχους ισότητας αποτύχει, επιστρέφεται false. Τέλος, εφόσον η equals δεν έχει ήδη επιστρέψει, τα αντικείμενα θεωρούνται ίσα και επιστρέφεται true. Γενικά, μέσω της equals δίνουμε τον ορισμό ισότητας μεταξύ των αντικειμένων της κλάσης.

Όπως γνωρίζουμε, ο έλεγχος της ισότητας μεταξύ πραγματικών γίνεται συνήθως προσεγγιστικά, δηλαδή αν η απόλυτη τιμή της διαφοράς δύο πραγματικών είναι μικρότερη από μία οριακή τιμή που συνήθως ονομάζεται epsilon, οι αριθμοί θεωρούνται ίσοι. Ωστόσο, μια τέτοια προσεγγιστική ισότητα δεν πληροί τη μεταβατική ιδιότητα, επομένως η συνάρτηση equals δεν μπορεί να βασιστεί σε προσεγγιστική ισότητα μεταξύ των μελών της. Αν χρειάζεστε μια προσεγγιστική ισότητα, εφοδιάστε την κλάση σας με κατάλληλη συνάρτηση.

```
public boolean aproximateEquals(Rectangle r) {
     if (equals(r)) {
          return true;
     }
     if (Math.abs(width - r.width) < 0.001 && Math.abs(length
     - r.length) < 0.001) {
          return true;
     }
     return false;
}
```

Τέλος, ας σημειωθεί ότι ο τύπος της παραμέτρου της equals είναι Object. Ωστόσο, η πραγματική παράμετρος μπορεί να είναι Rectangle. Πρόκειται επίσης για ένα θέμα που σχετίζεται με την κληρονομικότητα και εξηγείται στην ενότητα 13.

11.6.3 Η συνάρτηση hashCode

Τα αντικείμενα της κλάσης μας ενδέχεται να φορτωθούν σε συλλογές κατακερματισμού (hash Collections), όπως το HashMap ή το HashSet. Οι συλλογές αυτές χρειάζονται ένα κλειδί κατακερματισμού (hash-key) για κάθε αντικείμενο που αποθηκεύουν το οποίο χρησιμοποιούν ώστε να επιτυγχάνουν ταχύτερη προσπέλαση. Καθώς κάθε αντικείμενο είναι πιθανό κάποια στιγμή να αποθηκευτεί σε συλλογή κατακερματισμού, θα πρέπει κάθε αντικείμενο να είναι σε θέση να υπολογίζει την τιμή κατακερματισμού του. Αυτός ακριβώς είναι ο σκοπός της συνάρτησης hashCode, να επιστρέψει μια τιμή κατακερματισμού για το αντικείμενο που την καλεί.

Η ανάπτυξη της hashCode πρέπει υποχρεωτικά να υπακούει σε τρεις κανόνες:

1. Κάθε κλήση της που γίνεται κατά τη διάρκεια της εκτέλεσης μιας εφαρμογής θα πρέπει να επιστρέφει το ίδιο hashCode για το ίδιο αντικείμενο. Μεταξύ διαφορετικών εκτελέσεων της ίδιας εφαρμογής δεν είναι απαραίτητο να επιστρέφεται ο ίδιος κωδικός κατακερματισμού.
2. Δύο αντικείμενα για τα οποία η equals επιστρέφει true θα πρέπει να έχουν τον ίδιο κωδικό κατακερματισμού
3. Δύο αντικείμενα για τα οποία η equals επιστρέφει false δεν είναι υποχρεωτικό να επιστρέφουν διαφορετικό κωδικό κατακερματισμού. Είναι όμως χρήσιμο. Στην περίπτωση για παράδειγμα που μια συνάρτηση hashCode επιστρέφει 1 για κάθε αντικείμενο μιας κλάσης, τότε η προσπέλαση μέσα στις συλλογές κατακερματισμού θα μετατραπεί σε σειριακή.

Γενικότερα, μία καλή συνάρτηση κατακερματισμού έχει ως στόχο την ελαχιστοποίηση παραγωγής ίδιων κωδικών για διαφορετικά αντικείμενα.
Ακολουθεί μια απλή συνάρτηση κατακερματισμού για την κλάση Rectangle.

```
@Override
public int hashCode() {
     return Double.hashCode(length) + Double.hashCode(width);
}
```

Γενικά μια συνάρτηση hashCode επιστρέφει έναν int, η τιμή του οποίου είναι συνάρτηση των τιμών των μεταβλητών του αντικειμένου που καλεί την hashCode. Εφόσον, οι μεταβλητές του αντικειμένου είναι αντικείμενα και οι ίδιες, μπορούμε να αξιοποιήσουμε τη συνάρτηση hashCode της κλάσης που ανήκει η μεταβλητή. Στην περίπτωσή που οι μεταβλητές είναι θεμελιώδεις (native) τύποι μπορούμε να χρησιμοποιήσουμε τη στατική hashCode της αντίστοιχης κλάσης όπως κάναμε σε αυτό το παράδειγμα.

Όπως αναφέραμε προηγουμένως, η συνάρτηση κατακερματισμού που παρουσιάζουμε είναι αρκετά απλή. Πληροί μεν τους κανόνες της συνάρτησης κατακερματισμού αλλά δεν είναι ιδιαίτερα αποτελεσματική. Για παράδειγμα, δύο αντικείμενα Rectangle, το ένα με μήκος 2 και πλάτος 6 και το άλλο με μήκος 6 και πλάτος 2, θα παράξουν τον ίδιο κωδικό κατακερματισμού παρότι είναι διαφορετικά, δηλαδή η equals επιστρέφει false.

Αυτό βέβαια το πρόβλημα μπορεί να αντιμετωπιστεί αν πολλαπλασιάσουμε τη μία διάσταση με έναν ακέραιο. Πάντως, όλα τα σύγχρονα IDE παράγουν αυτόματα τη συνάρτηση hashCode. Για παράδειγμα, το NetBeans, για την κλάση Rectangle, παραγάγει την ακόλουθη συνάρτηση κατακερματισμού:

```
@Override
public int hashCode() {
     int hash = 3;
     hash      =      53      *      hash      +      (int)
     (Double.doubleToLongBits(this.length)                  ^
     (Double.doubleToLongBits(this.length) >>> 32));
```

```
        hash = 53 * hash + (int)
        (Double.doubleToLongBits(this.width) ^
        (Double.doubleToLongBits(this.width) >>> 32));
        return hash;
    }
```

Η doubleToLongBits(length) επιστρέφει έναν long που όταν η δυαδική του αναπαράσταση μετατραπεί σε double, το αποτέλεσμα θα είναι ακριβώς ίσο με length. Στο αποτέλεσμα αυτό γίνεται μη προσημασμένη δεξιά διολίσθηση 32 θέσεων και αυτό που επιστρέφεται γίνεται αποκλειστικό or πάλι με την doubleToLongBits(length). Το νέο αποτέλεσμα το μετατρέπουμε σε int και προσθέτουμε έναν ακέραιο. Οι σταθερές 3 και 53, πιθανότατα να μην είναι ίδιες σε επαναλαμβανόμενες αιτήσεις προς το NetBeans να μας φτιάξει τη συνάρτηση κατακερματισμού της κλάσης Rectangle.

Περισσότερες πληροφορίες για το πώς μπορείτε να δημιουργείτε αποτελεσματικότερες συναρτήσεις κατακερματισμού μπορείτε να βρείτε στα βιβλία των Adekunle [2] και Aumasson, et all. [3].
Ακολουθεί η τελική έκδοση της κλάσης Rectangle στον κώδικα 11.8.

```
public class Rectangle {

    private double length, width;

    public Rectangle(double length, double width) {
        if (length <= 0 || width <= 0) {
            throw new IllegalArgumentException();
        }
        this.length = length;
        this.width = width;
    }

    public Rectangle(double length) {
        this(length, length);
    }

    public Rectangle() {
        this.length = 2;
        this.width = 2;
    }

    public double area() {
        return length * width;
    }

    public double perimeter() {
        return 2 * (this.length + this.width);
    }

    public double diagonal() {
        return Math.sqrt(Math.pow(this.length, 2) + Math.pow(this.width,
2));
    }

    public double getLength() {
        return this.length;
    }

    public void setLength(double length) {
        if (length <= 0) {
            throw new IllegalArgumentException();
        }
        this.length = length;
```

```
    }

    public double getWidth() {
        return this.width;
    }

    public void setWidth(double width) {
        if (width <= 0) {
            throw new IllegalArgumentException();
        }
        this.width = width;
    }

    @Override
    public String toString() {
        return "Rectangle{" + "length=" + length + ", width=" + width +
'}';
    }

    @Override
    public int hashCode() {
        return Double.hashCode(length) + Double.hashCode(width);
    }
    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (obj == null) {
            return false;
        }
        if (getClass() != obj.getClass()) {
            return false;
        }
        final Rectangle other = (Rectangle) obj;
        if (Double.compare(width, other.width) != 0) {
            return false;
        }
        return Double.compare(length, other.length) == 0;
    }

    public boolean aproximateEquals(Rectangle r) {
        if (equals(r)) {
            return true;
        }
        return Math.abs(width - r.width) < 0.001 && Math.abs(length -
r.length) < 0.001;
    }
}
```

Κώδικας 11.8 *Υπόδειγμα ολοκληρωμένης κλάσης*

11.7 Απαριθμήσιμοι τύποι

Μερικές φορές στα προγράμματά μας χρειαζόμαστε ένα ή περισσότερα σύνολα από σταθερές. Για παράδειγμα, έστω ότι θέλουμε να αναπαραστήσουμε με σταθερές κάθε ημέρα της εβδομάδας έτσι ώστε να μην αναφερόμαστε σε κυριολεκτικούς ακέραιους που κάνουν το πρόγραμμά μας δυσανάγνωστο.
Για την καλύτερη ικανοποίηση τέτοιων αναγκών, η Java προσφέρει τον τύπο enum. Στον κώδικα 11.9, παρουσιάζουμε τη δήλωση ενός απαριθμήσιμου τύπου ορισμού των ημερών της εβδομάδας.

```
public enum Day {
    MONDAY,
    THUSDAY,
    WEDNESDAY,
    THURSDAY,
    FRIDAY,
    SATURDAY,
    SUNDAY;
}
```

Κώδικας 11.9 *Απαριθμήσιμος τύπος για τις ημέρες της εβδομάδας*

Όπως φαίνεται στον κώδικα 11.9, η δήλωση απαριθμήσιμου τύπου γίνεται με χρήση της λέξης-κλειδί enum. Κάθε μια από τις μεταβλητές MONDAY έως FRIDAY είναι και μία σταθερά. Έτσι, τα αντίστοιχα αναγνωριστικά εκφράζονται με κεφαλαία ώστε να πληρούνται οι συμβάσεις ονοματολογίας των σταθερών. Επιπλέον, οι μεταβλητές αυτές είναι εξ ορισμού public και static. Από τη στιγμή που έχουμε διαθέσιμο τον τύπο Day, μπορούμε να τον χρησιμοποιήσουμε στον κώδικά μας, καταρχάς δηλώνοντας μεταβλητές τύπου Day, π.χ.

```
Day d=Day.FRIDAY;
```

Στη συνέχεια μπορούμε να χρησιμοποιήσουμε κανονικά τη μεταβλητή d, όπως για παράδειγμα δείχνει ο κώδικας που ακολουθεί:

```
switch (d) {
     case MONDAY:
          System.out.println("First day of the Week");
          break;
     case THUSDAY:
          System.out.println("Second day of the Week");
          break;
     case WEDNESDAY:
          System.out.println("It's a long Week");
          break;
     case THURSDAY:
          System.out.println("I fill tired");
          break;
     case FRIDAY:
          System.out.println("Last day of the Working Week");
          break;
     case SATURDAY:
          System.out.println("It's a terrific day");
          break;
     case SUNDAY:
          System.out.println("Rest today");
          break;
}
```

Ας δούμε όμως μερικά μυστικά αυτού του τύπου. Καταρχάς, πρόκειται για εξειδικευμένη κλάση. Οι δε τιμές MONDAY έως FRIDAY της Day δεν είναι παρά αντικείμενα της Day που έχουν δημιουργηθεί με τον default δημιουργό. Βεβαίως, όπως θα δούμε αμέσως παρακάτω, μπορούμε να ορίσουμε και δικούς μας δημιουργούς. Επιπλέον, τα αντικείμενα αυτά είναι public, static και final παρότι δεν έχουν δηλωθεί σαφώς ως τέτοια. Ο τύπος ονομάζεται απαριθμήσιμος καθώς τα αντικείμενα-τιμές αντιστοιχίζονται σε μία ακέραιη τιμή. Οι εξ ορισμού τιμές είναι 0,1,2,..,ν, για ν+1 τιμές ενός απαριθμήσιμου τύπου. Έτσι η τιμή της MONDAY είναι 0, της THUSDAY 1, κοκ, έως την SUNDAY με τιμή 6. Ο τύπος μάλιστα διαθέτει μια μέθοδο, την ordinal, που μας επιστρέφει την τιμή ενός απαριθμήσιμου αντικειμένου, π.χ. ο κώδικας

```
Day d = Day.FRIDAY;
System.out.println(d.ordinal());
```

θα τυπώσει 4.
Επιπλέον, η μέθοδος toString θα επιστρέψει σε μορφή String την ονομασία του αντικειμένου, δηλαδή ο κώδικας

```
Day d = Day.FRIDAY;
System.out.println(d);
```

εμφανίζει FRIDAY.
Επίσης, ο τύπος διαθέτει τη στατική μέθοδο values που επιστρέφει τα αντικείμενα σε πίνακα, δηλαδή η πρόταση

```
System.out.println(Arrays.asList(Day.values()));
```

θα εμφανίσει

```
[MONDAY, THUSDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY,
SUNDAY]
```

Τέλος, να επισημάνουμε πως ο εξ ορισμού δημιουργός του τύπου enum είναι ιδιωτικός ώστε να μην έχει ο χρήστης του την δυνατότητα να δημιουργήσει νέα αντικείμενα και να αλλοιώσει τον χαρακτήρα του τύπου.
Ας δούμε όμως και ένα άλλο παράδειγμα, όπου η αρίθμηση δεν ακολουθεί την τυπική περίπτωση από 0..ν. Στον κώδικα 11.10 δίνουμε έναν απαριθμήσιμο τύπο που αντιπροσωπεύει τους βαθμούς των καρτών μιας τράπουλας, έτσι ώστε το 2 της τράπουλας να έχει αρίθμηση 2, το 3 να έχει αρίθμηση 3 και το μοτίβο αυτό να ακολουθείται έως το 10, ενώ ο Βαλές έχει αρίθμηση 11, η Ντάμα 12, ο Ρήγας 13 και ο Άσος 14.

```
import java.util.Arrays;

public enum CardRank {

    TWO(2),
    THREE(3),
    FOUR(4),
    FIVE(5),
    SIX(6),
    SEVEN(7),
    EIGHT(8),
    NINE(9),
    TEN(10),
    JACK(11),
    QUEEN(12),
    KING(13),
    ACE(14);

    private final int cardValue;

    private CardRank(int value) {
        this.cardValue = value;
    }

    public int getCardValue() {
        return cardValue;
    }
}
```

Κώδικας 11.10 *Ο απαριθμήσιμος τύπος CardRank*

Καθώς δεν μας διευκολύνει η εξ ορισμού αρίθμηση, έχουμε φτιάξει έναν δημιουργό με τον οποίο δίνουμε τιμή στην cardValue την οποία μπορούμε να χρησιμοποιούμε σαν τιμή των μεταβλητών του τύπου. Κατά τη δημιουργία των αντικειμένων TWO έως ACE καλείται τώρα ο νέος δημιουργός και όπου παραστεί ανάγκη μπορούμε να πάρουμε ως αρίθμηση των αντικειμένων την cardValue μέσω της getCardValue. Προσοχή όμως, η τιμή αυτή δεν αντικαθιστά την τιμή που επιστρέφει η ordinal. Έτσι, η μεν CardRank.NINE.getCardValue() θα επιστρέψει 9, η δε CardRank.NINE.ordinal() θα επιστρέψει 7.

Ένα άλλο σημαντικό στοιχείο στον κώδικα 11.10 είναι πως ο δημιουργός είναι υποχρεωτικά ιδιωτικός. Αυτή είναι η τυπική περίπτωση στους απαριθμήσιμους τύπους. Κάνοντας τον δημιουργό ιδιωτικό, μπορεί να χρησιμοποιηθεί μόνο από την κλάση για την δημιουργία των απαιτούμενων τιμών, δηλαδή στο παράδειγμά μας, για τη δημιουργία των σταθερών TWO έως ACE. Αν όμως είχαμε ορίσει έναν δημιουργό δημόσιο, τότε οποιοσδήποτε θα μπορούσε να δημιουργήσει τέτοια αντικείμενα με αποτέλεσμα να καταστραφεί η έννοια του απαριθμήσιμου τύπου. Φανταστείτε να δημιουργήσει κανείς τιμή FRIDAY τύπου CardRank. Αν προσπαθήσετε να δηλώσετε δημιουργό σε τύπο enum με δημόσια ή προστατευμένη πρόσβαση, ο μεταγλωττιστής θα διαμαρτυρηθεί. Αν πάλι δεν δηλώσετε προσδιοριστή πρόσβασης, τότε η πρόσβαση δεν θα είναι πρόσβαση πακέτου όπως ισχύει για τις τυπικές κλάσεις αλλά αντίθετα θα καθορισθεί άρρητα ως ιδιωτική.

Έχοντας τον δημιουργό ιδιωτικό έχουμε εξασφαλίσει πως μέσα στο πρόγραμμά μας δεν υπάρχουν άλλα αντικείμενα τύπου CardRank πέραν των δηλωθέντων στον ίδιο τον τύπο. Επομένως, κάθε βαθμός κάρτας στο πρόγραμμά μας είναι μοναδικός, δηλαδή υπάρχει ένα μόνο αντικείμενο TWO, ένα μόνο αντικείμενο THREE και ούτω καθεξής. Σαν συνέπεια, μπορούμε να ελέγχουμε για ισότητα τα αντικείμενα CardRank με χρήση του τελεστή ισότητας. Παρόλο που ο τύπος περιλαμβάνει τη συνάρτηση equals, σε αυτήν την περίπτωση δεν είναι αναγκαία η χρήση της. Αν υποθέσουμε πως έχουμε δύο αναφορές σε ένα αντικείμενο CardRank. Αν και οι δύο αναφορές έχουν ως περιεχόμενο το αντικείμενο TWO ας πούμε, τότε θα αναφέρονται στην ίδια διεύθυνση, μιας και υπάρχει μόνο ένα τέτοιο στιγμιότυπο στην εφαρμογή μας. Επομένως, η σύγκριση με τον τελεστή ισότητας είναι έγκυρη και με δεδομένο πως είναι ταχύτερη από την κλήση της equals την προτιμούμε.

11.8 Εμφωλευμένες κλάσεις

Έχουμε εξηγήσει πως σε ένα αρχείο java μπορεί να περιέχεται μία μόνο δημόσια κλάση και μάλιστα απαιτείται το όνομα του αρχείου να είναι ίδιο με το όνομα της κλάσης (ενότητα 2.2). Τίποτα όμως δεν μας εμποδίζει να τοποθετήσουμε σε ένα αρχείο περισσότερες από μία μη δημόσιες κλάσεις. Για παράδειγμα, ο κώδικας

```
class NonPublic { }

public class NonPublicDemo {

     public static void main(String[] args) {
          NonPublic nP=new NonPublic();
     }
}
```

έστω ότι περιλαμβάνει την κλάση NonPublic στο ίδιο αρχείο με τη δημόσια κλάση NonPublicDemo. Σε αυτό το επίπεδο επιτρέπεται μόνο η πρόσβαση πακέτου για την κλάση NonPublic.
Υπάρχει, επιπλέον η δυνατότητα να ορίσουμε εμφωλευμένες κλάσεις (nested class). Οι εμφωλευμένες κλάσεις είναι δύο κατηγοριών, οι στατικές και οι μη στατικές που ονομάζονται εσωτερικές (inner). Ο κώδικας που ακολουθεί επιδεικνύει τη χρήση εσωτερικής κλάσης.

```
public class Outer {

     private int k = 1;

     class Inner {

          int getK() {
               return k;
          }
```

```
        }

        public static void main(String[] args) {
              Outer outer = new Outer();
              Outer.Inner iCObject = outer.new Inner();
              System.out.println(outer.k + " " + iCObject.getK());
        }
  }
```

Οι εσωτερικές κλάσεις δέχονται όλους τους ρυθμιστές πρόσβασης, δηλαδή μία εσωτερική κλάση μπορεί να χαρακτηριστεί ως δημόσια, ιδιωτική, προστατευμένη (ενότητα 13.1.2) ή απλώς να διατηρήσει την πρόσβαση πακέτου. Ένα πλεονέκτημα των εσωτερικών κλάσεων είναι ότι έχουν πρόσβαση στα μέλη της κλάσης που ανήκουν ακόμη και όταν αυτά είναι ιδιωτικά. Όπως φαίνεται στον κώδικα, η κλάση Inner προσπελαύνει το ιδιωτικό μέλος k της Outer. Για να δημιουργηθεί αντικείμενο εσωτερικής κλάσης, πρέπει πρώτα να δημιουργηθεί αντικείμενο της περιβάλλουσας μέσα από το οποίο μπορεί να κληθεί ο τελεστής new όπως επιδεικνύει ο παραπάνω κώδικας. Οι εσωτερικές κλάσεις μπορούν να αναπτυχθούν και τοπικά σε επίπεδο συνάρτησης.

Ένας άλλος τύπος εσωτερικής κλάσης είναι η ανώνυμη κλάση (Anonymous class). Θα παρουσιάσουμε την ανώνυμη κλάση στην ενότητα 14.4.5.

Όπως όλα τα μέλη μιας κλάσης, έτσι και τα μέλη-κλάσεις μπορούν να είναι στατικά. Ακολουθεί παράδειγμα χρήσης εμφωλευμένης στατικής κλάσης (nested static class):

```
public class Outer {

     private int k = 1;

     static class NestedStatic {

          int getK(Outer outer) {
               return outer.k;
          }
     }

     public static void main(String[] args) {
          Outer.NestedStatic nS = new NestedStatic();
          Outer outer = new Outer();
          System.out.println(outer.k + " " + nS.getK(outer));
     }
}
```

Η κλάση NestedStatic έχει δηλωθεί ως στατική. Γενικά τα στατικά μέλη δεν έχουν πρόσβαση στα μη στατικά μέλη μιας κλάσης. Ο γενικός αυτός κανόνας ισχύει και για τις κλάσεις. Ένα στατικό μέλος μπορεί να εκτελεστεί αν είναι μέθοδος ή να δημιουργηθεί αν είναι κλάση χωρίς να συσχετίζεται υποχρεωτικά με κάποιο στιγμιότυπο της κλάσης. Επομένως, δεν είναι δυνατόν να προσπελάσει άμεσα τα μη στατικά μέλη. Μπορεί βέβαια η προσπέλαση να επιτευχθεί μέσα από ένα αντικείμενο της κλάσης. Έτσι, η getK εδώ λαμβάνει ως παράμετρο ένα αντικείμενο outer του οποίου εμφανίζει την μεταβλητή k. Όπως οι εσωτερικές κλάσεις, έτσι και οι εμφωλευμένες στατικές μπορούν να συνταχθούν με οποιονδήποτε προσδιοριστή πρόσβασης.

11.9 Λυμένες Ασκήσεις

Παρουσιάζουμε εδώ μια σειρά από λυμένες ασκήσεις που θα βοηθήσουν τον αναγνώστη να εξοικειωθεί με τη λογική της ενσωμάτωσης.

11.9.1 Η κλάση Person

Να υλοποιηθεί η κλάση Person. Κάθε Person διαθέτει όνομα, επώνυμο και φύλο. Για το φύλο χρησιμοποιήστε int έτσι ώστε το 0 να αναπαριστά το γυναικείο φύλλο και το 1 το ανδρικό. Ορίστε τις αναγκαίες συναρτήσεις.

Λύση

```
public class Person {

    private String fName;
    private String sName;
    private int sex;

    public Person(String fName, String sName, int sex) {
        this.fName = fName;
        this.sName = sName;
        this.sex = sex;
    }

    public String getfName() {
        return fName;
    }

    public void setfName(String fName) {
        this.fName = fName;
    }

    public String getsName() {
        return sName;
    }

    public void setsName(String sName) {
        this.sName = sName;
    }

    public int getSex() {
        return sex;
    }

    public void setSex(int sex) {
        this.sex = sex;
    }

    @Override
    public int hashCode() {
        int hash = 5;
        hash = 89 * hash + this.fName.hashCode();
        hash = 89 * hash + this.sName.hashCode();
        hash = 89 * hash + this.sex;
        return hash;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (obj == null) {
            return false;
        }
        if (getClass() != obj.getClass()) {
            return false;
        }
```

```
            final Person other = (Person) obj;
            if (this.sex != other.sex) {
                return false;
            }
            if (!this.fName.equals(other.fName)) {
                return false;
            }
            return this.sName.equals(other.sName);
        }

        @Override
        public String toString() {
            return "Person{" + "fName=" + fName + ", sName=" + sName + ", sex="
    + (sex==1?"M":"F") + '}';
        }
    }
```

Κώδικας 11.11 *Η κλάση Person*

Στον κώδικα 11.11 παρατίθεται η κλάση Person. Πρόκειται για απλή εφαρμογή της ενσωμάτωσης. Παρόμοιο αποτέλεσμα μπορεί να παραχθεί αυτόματα από το NetBeans ως εξής: Δημιουργήστε μια κλάση Person και προσθέστε τις μεταβλητές στιγμιότυπου. Μεταβείτε στον editor του NetBeans στην κλάση. Με δεξί κλικ ανοίγει ένα μενού. Επιλέξτε Insert Code. Από το υπομενού που θα ανοίξει επιλέξτε σταδιακά την αυτόματη δημιουργία δημιουργών, αναγνωστών, ρυθμιστών, μεθόδου equals, μεθόδου hashcode και μεθόδου toString. Εντοπίστε και μελετήστε τις διαφορές μεταξύ του κώδικα που παράγει το NetBeans και του κώδικα 11.11.

11.9.2 Η κλάση MyInteger

Υλοποιήστε την κλάση MyInteger που περιλαμβάνει τις συναρτήσεις που περιγράφονται παρακάτω και όλες τις αναγκαίες συναρτήσεις.

```
public class MyInteger {
     private int value;
     public MyInteger();
     public MyInteger(int k);
     public MyInteger add(MyInteger i);
     public MyInteger subtract(MyInteger i);
     public MyInteger multiply(MyInteger i);
     public MyInteger divide(MyInteger i);
     public String toString();
     public boolean equals(Object obj);
     public int hashCode();
};
```

Να υπερφορτωθούν οι μέθοδοι setValue, add, subtract, multiply, divide ώστε να δέχονται παραμέτρους τύπου Integer και τύπου int.

Λύση

```
public class MyInteger {

    private int value;

    public MyInteger() {
        value = 0;
    }

    public MyInteger(int k) {
        value = k;
```